Project Topic: Emotion-Aware Support Companion

GRAD 695 52 B- Research Methodology and Writing

Harrisburg University of Science and Technology

Submitted to: Abdel Ejnioui, Ph. D

Abstract

There are growing instances of emotion-conscious interfaces in education, customer service applications, and wellbeing apps, but most of them have no calibrated responses, clear consent flows, or analysis other than accuracy metrics. In this project, we can fill in such gaps by integrating technical prototyping with user-centred protective measures, as well as rigorous assessment, which complies with the objectives of the course in research design, instrument validity, and ethical practice.

Keywords: *Emotion-Aware Support Companion*

Table of Contents

Abstract 2
Introduction 7
Methods & Expected Deliverables 7
Use Case Documentation 7
System Analysis – Entity, Boundary, and Control Objects 21
Entity Objects 21
Boundary Objects 23
Control Objects 25
UML Class / Object Diagram 28
UML Diagram 28
Objects Grouped into Subsystems 30
Subsystem Descriptions 31
User Interface Subsystem 31
Emotion & Safety Subsystem 32
Consent & Privacy Subsystem 32
Privacy Policy Engine 32
Data Layer 33
Audit & Logging Subsystem 33
Subsystem Identification (Heuristics Reference) 33
Privacy Tactic: Policy-Driven Data Retention and Deletion 36
Expanded Class/Object Model 39
Identification of Persistent Entity Objects 44
Relational Database Mapping 46
Mapping Associations 50
Many-to-Many Mapping 51
API Design 57
Entity Summary 57
Methods 57
REST API Design (CRUD) 58
GraphQL API Design 62

Type Definition 62
Queries 63
Mutations 63
Conclusion 65
Perspective Thoughts and Reflections for Future Directions 65
References 67

List of Tables

Table 1: Showing UC01 – InitiateConversation 8
Table 2: showing UC02 – SendMessageForAnalysis 10
Table 3 showing UC03 – GenerateSupportiveResponse 11
Table 4 showing UC04 – ProvideCrisisSupportSuggestion 13
Table 5 showing UC05 – RequestClarification 14
Table 6 UC06 – ViewConsentInformation 16
Table 7 showing UC07 – ManagePrivacyPreferences 17
Table 8 showing UC08 – EndConversationSession 18
Table 9: Showing UC09 – ViewConversationHistory 20
Table 10: Showing UC10 – ProvideSessionFeedback 21
Table 11 Showing Table – Entity objects for the Emotion-Aware Support Companion 23
Table 12 Showing Table – Boundary objects for the Emotion-Aware Support Companion 24
Table 13 Showing Control Objects 27
Table 14: Showing Subsystem Descriptions 30
Table 15 Showing System Quality Attribute: Privacy 35
Table 16 Showing Subsystem Responsibility Table 39
Table 17 Showing Persistent vs Non-Persistent Entities 46
Table 18 Showing Mapping Summary Table 56
Table 19 Showing 57
Table 20 Showing Methods 57

List of Figures

Figure 1 Showing UML Class / Object Diagram..... 28
Figure 2 Showing UML Diagram 29
Figure 3 Showing UML Component Diagram – Subsystem Decomposition (Emotion-Aware Support Companion). 34
Figure 4 Showing UML component diagram illustrating the policy-driven data retention and deletion tactic. The diagram shows how consent withdrawal triggers privacy policy evaluation and enforcement across subsystems, ensuring timely deletion or anonymisation of sensitive 37

Introduction

This project proposes an Emotion-Aware Support Companion that can recognise, interpret, and respond to users' emotional states during text-based interactions (Li, 2025). Drawing on affective computing and natural language processing, the system will infer sentiment and basic emotions from lexical choice, syntax, and dialogue context, then generate supportive, ethically-constrained responses. The prototype will address ethical considerations through transparency, user agency, and autonomy (Li, 2025). We expect to show how simple, explainable models coupled with close conversational architecture can advance the perception of empathy and user satisfaction with a significant benefit of not making overly dramatic clinical assertions.

Methods & Expected Deliverables

i. Design: Brief requirements, anticipated conversation flows, risk and consent measures.
ii. Prototype: Text-based interface with sentiment or emotion classifier; rule-based safety guard rails.
iii. Evaluation: Small user study (n≈10-15) on usability, trust, and perceived empathy (Likert scales + thematic analysis).
iv. Deliverables: Project proposal, annotated bibliography, IRB/ethics checklist, prototype walkthrough, final report covering methods, findings, and discussion of limitations.

Use Case Documentation

Field	**Description**
Use case name	InitiateConversation
Participating actors	User, EmotionAwareSupportCompanion

Initiated by	User
Communicates with	EmotionAwareSupportCompanion
Flow of events	1. The User opens the Emotion-Aware Support Companion application or website.2. EmotionAwareSupportCompanion displays the home screen with a brief description of the service.3. The User selects the “Start Conversation” function.4. EmotionAwareSupportCompanion displays a consent notice and a short disclaimer about limitations and non-clinical status.5. The User reads and accepts the consent notice.6. EmotionAwareSupportCompanion creates a new conversation session and opens the chat interface for the User.
Entry condition	• EmotionAwareSupportCompanion is running and reachable .• The User has network connectivity.
Exit conditions	• A new active conversation session has been created. • The chat interface is displayed and ready to accept the first message from the User.
Quality requirements	• The home screen appears within 2 seconds. • The consent notice must be clearly readable on both mobile and desktop screens.

Table 1: Showing UC01 – InitiateConversation

Field	**Description**
Use case name	SendMessageForAnalysis
Participating actors	User, EmotionAwareSupportCompanion
Initiated by	User
Communicates with	EmotionAnalysisComponent (inside EmotionAwareSupportCompanion)
Flow of events	1. The User enters a text message describing their feelings or situation in the chat interface. 2. The User selects the “Send” control. 3. EmotionAwareSupportCompanion receives the message and associates it with the current conversation session. 4. EmotionAwareSupportCompanion forwards the message text to the EmotionAnalysisComponent. 5. EmotionAwareSupportCompanion logs the message and timestamp according to the User’s privacy preferences.
Entry condition	• There is an active conversation session for the User. • The chat interface is visible.
Exit conditions	• The message has been stored in the conversation history.

	• The EmotionAnalysisComponent has been requested to analyse the message.
Quality requirements	• The message must be forwarded for analysis within 1 second. • No truncation or modification of user text occurs during transfer.

Table 2: showing UC02 – SendMessageForAnalysis

Field	**Description**
Use case name	GenerateSupportiveResponse
Participating actors	EmotionAwareSupportCompanion, User
Initiated by	EmotionAwareSupportCompanion
Communicates with	User
Flow of events	1. EmotionAwareSupportCompanion receives an emotion label and confidence score from the EmotionAnalysisComponent. 2. EmotionAwareSupportCompanion selects a response strategy based on the emotion label, recent conversation context, and safety rules. 3. EmotionAwareSupportCompanion composes a supportive response using templates and contextual information.

	4. EmotionAwareSupportCompanion checks the composed response against rule-based safety guard rails (e.g., no diagnoses, no promises, no harmful advice). 5. EmotionAwareSupportCompanion displays the supportive response in the chat interface for the User .6. EmotionAwareSupportCompanion stores the response as part of the conversation history.
Entry condition	• The User's latest message has been analysed. • A valid emotion label has been produced.
Exit conditions	• A supportive, context-appropriate response is shown to the User.• The response is recorded in the conversation history.
Quality requirements	• The supportive response must appear within 1.5 seconds after the User sends the message. • All responses must comply with pre-defined ethical and safety constraints.

Table 3 showing UC03 – GenerateSupportiveResponse

Field	**Description**
Use case name	ProvideCrisisSupportSuggestion

Participating actors	EmotionAwareSupportCompanion, User
Initiated by	EmotionAwareSupportCompanion
Communicates with	User
Flow of events	1. EmotionAwareSupportCompanion detects high-risk language or patterns (e.g., self-harm, harm to others, severe distress) in the User's message. 2. EmotionAwareSupportCompanion flags the message as potential crisis using the safety rules engine. 3. EmotionAwareSupportCompanion selects a crisis-oriented supportive message template instead of a normal supportive response. 4. EmotionAwareSupportCompanion displays a neutral, non-judgemental message encouraging the User to contact trusted people or professional/emergency services. 5. EmotionAwareSupportCompanion optionally displays crisis hotline details or local support resources if configured. 6. EmotionAwareSupportCompanion stores a CrisisFlag associated with the message and session.

Entry condition	• A User message has been analysed and marked as high-risk by the safety rules. • The conversation session is active.
Exit conditions	• A crisis support suggestion has been displayed to the User. • The corresponding CrisisFlag has been recorded.
Quality requirements	• Crisis checks must be executed for every user message. • Crisis suggestions must never provide clinical diagnoses or instructions for self-harm.

Table 4 showing UC04 – ProvideCrisisSupportSuggestion

Field	**Description**
Use case name	RequestClarification
Participating actors	EmotionAwareSupportCompanion, User
Initiated by	EmotionAwareSupportCompanion
Communicates with	User
Flow of events	1. EmotionAwareSupportCompanion receives an emotion analysis result with confidence below a predefined threshold or with conflicting

	indicators.2. EmotionAwareSupportCompanion constructs a short clarification question in neutral language. 3. EmotionAwareSupportCompanion displays the clarification question to the User in the chat interface. 4. The User may provide additional detail or rephrase their message. 5. EmotionAwareSupportCompanion forwards the new message to the EmotionAnalysisComponent and continues with GenerateSupportiveResponse.
Entry condition	• A User message has been received. • The emotion analysis confidence is below the configured threshold.
Exit conditions	• Either a clarifying message has been received and re-analysed, or the User has chosen not to respond. • The clarification interaction is recorded in the conversation history.
Quality requirements	• Clarification prompts must be polite and non-judgemental. • The system must not repeatedly ask for clarification if the User does not respond.

Table 5 showing UC05 – RequestClarification

Field	**Description**
Use case name	ViewConsentInformation

Participating actors	User, EmotionAwareSupportCompanion
Initiated by	User
Communicates with	ConsentModule (inside EmotionAwareSupportCompanion)
Flow of events	1. The User selects the “Data & Consent” option from the home screen or settings menu.2. EmotionAwareSupportCompanion retrieves the current consent and data-use policy text.3. EmotionAwareSupportCompanion displays the consent information in plain language, including limitations and emergency disclaimers.4. The User scrolls through and reviews the content.5. The User closes the view or indicates that they have understood the information.
Entry condition	• The User has opened the application or is in an active session. • Consent text has been configured in the system.
Exit conditions	• The User has been given an opportunity to review the consent and data-use information. • EmotionAwareSupportCompanion may log that the consent information was viewed.
Quality requirements	• Consent text must be clearly legible without horizontal scrolling. • The reading level should be appropriate for non-technical users.

Table 6 UC06 – ViewConsentInformation

Field	**Description**
Use case name	ManagePrivacyPreferences
Participating actors	User, EmotionAwareSupportCompanion
Initiated by	User
Communicates with	PrivacySettingsModule
Flow of events	1. The User opens the “Privacy Settings” view from the home screen or menu. 2. EmotionAwareSupportCompanion retrieves the current PrivacySettings associated with the User. 3. EmotionAwareSupportCompanion displays options such as data-retention period, storage of conversation history, and anonymisation of logs. 4. The User modifies one or more privacy options. 5. The User confirms the changes. 6. EmotionAwareSupportCompanion validates the new settings and stores them securely.

Entry condition	• The User is identified for the current session (even if pseudonymously). • Privacy settings functionality is enabled.
Exit conditions	• Updated PrivacySettings are stored for the User. • Future logging and retention behaviour follow the new settings.
Quality requirements	• Changes must be persisted within 1 second. • Privacy settings must be stored using appropriate encryption and access control.

Table 7 showing UC07 – ManagePrivacyPreferences

Field	**Description**
Use case name	EndConversationSession
Participating actors	User, EmotionAwareSupportCompanion
Initiated by	User
Communicates with	SessionManager (inside EmotionAwareSupportCompanion)
Flow of events	1. The User selects the “End Session” control or exits through a dedicated button in the chat interface.

	2. EmotionAwareSupportCompanion displays a brief closing message and a reminder that it is not a crisis service. 3. SessionManager finalises the conversation according to the User's PrivacySettings (e.g., full log, partial summary, or minimal record). 4. EmotionAwareSupportCompanion marks the conversation session as closed. 5. EmotionAwareSupportCompanion returns the User to the home screen.
Entry condition	• There is an active conversation session. • The chat interface is open.
Exit conditions	• The session has status "closed". • No further messages are accepted for the closed session.
Quality requirements	• Session closure must complete within 1 second. • No user messages sent after closure are stored under the closed session.

Table 8 showing UC08 – EndConversationSession

Field	**Description**
Use case name	ViewConversationHistory
Participating actors	User, EmotionAwareSupportCompanion

Initiated by	User
Communicates with	SessionHistoryModule
Flow of events	1. The User selects the “Past Sessions” option from the home screen or menu. 2. EmotionAwareSupportCompanion retrieves a list of ConversationSessions that are allowed to be stored under the User’s PrivacySettings. 3. EmotionAwareSupportCompanion displays the list of sessions with dates and short summaries. 4. The User selects a specific session to view. 5. EmotionAwareSupportCompanion displays the selected conversation history or a summary of that session.
Entry condition	• The User has at least one stored ConversationSession. • The User has permitted the storage of session history.
Exit conditions	• The selected conversation history or summary is displayed to the User. • No modifications are made to the original records.
Quality requirements	• The list of past sessions must be retrieved within 3 seconds under normal load.

	• Histories must be clearly labelled with date and time information.

Table 9: Showing UC09 – ViewConversationHistory

Field	**Description**
Use case name	ProvideSessionFeedback
Participating actors	User, EmotionAwareSupportCompanion
Initiated by	User (optionally prompted by EmotionAwareSupportCompanion)
Communicates with	FeedbackModule
Flow of events	1. After EndConversationSession, EmotionAwareSupportCompanion invites the User to provide feedback, or the User selects “Give Feedback” from the menu. 2. EmotionAwareSupportCompanion displays a short feedback form with Likert-scale questions on usability, trust, and perceived empathy, and an optional free-text comment box. 3. The User fills in the ratings and, optionally, a comment. 4. The User submits the feedback form.

	5. FeedbackModule stores a FeedbackEntry in an anonymised or pseudonymised way. 6. EmotionAwareSupportCompanion thanks the User and returns to the home screen.
Entry condition	• A conversation session has ended, or the User has chosen to provide feedback. • Feedback functionality is enabled.
Exit conditions	• A FeedbackEntry has been stored for later evaluation. • The User is returned to the home screen or exits the application.
Quality requirements	• The feedback form should require no more than 1 minute to complete. • Feedback must not expose sensitive conversation content when anonymity is promised.

Table 10: Showing UC10 – ProvideSessionFeedback

System Analysis – Entity, Boundary, and Control Objects

Entity Objects

Entity	Description
UserProfile	Represents an individual or pseudonymous user interacting with the EmotionAwareSupportCompanion. A UserProfile stores an identifier, language or locale preferences, and a link to PrivacySettings. A UserProfile may be associated with many ConversationSessions over time.

ConversationSession	Represents a single interaction between a UserProfile and the EmotionAwareSupportCompanion. A ConversationSession includes a start time, an end time, a status (open or closed), and a short summary. A ConversationSession aggregates Messages, EmotionRecords, CrisisFlags, and may include a FeedbackEntry.
Message	Represents a single text exchange in a ConversationSession. A Message may be sent by the User or by the EmotionAwareSupportCompanion. A Message contains the text content, a timestamp, a sender type, and a reference to its ConversationSession. User Messages may be associated with EmotionRecords and CrisisFlags.
EmotionRecord	Represents the interpreted emotional state of a Message. An EmotionRecord stores an emotion label (sad, anxious, neutral, etc.), a confidence score, and a derived risk level. EmotionRecords are created by the emotion analysis process and used by control objects to generate supportive responses or crisis suggestions.
ConsentRecord	Represents the User's consent decision. A ConsentRecord includes the consent version, the time at which consent was given, and optionally the time at which consent was withdrawn. A UserProfile may have multiple ConsentRecords over time as policies change.
PrivacySettings	Represents the User's choices regarding data retention and anonymity. PrivacySettings specify whether ConversationSessions are stored, how

	long they are retained, and whether logs are anonymised. PrivacySettings affect how Messages, EmotionRecords, and FeedbackEntries are stored or deleted.
FeedbackEntry	Represents the User's evaluation of a ConversationSession. A FeedbackEntry includes ratings for usability, trust, and perceived empathy, as well as optional comments. A FeedbackEntry is linked to a single ConversationSession and is used to improve the overall system experience.
CrisisFlag	Represents an indication that a Message or ConversationSession contains potential crisis content. A CrisisFlag records the type of crisis cue, the time it was detected, and whether the issue has been reviewed or resolved.

Table 11 Showing Table – Entity objects for the Emotion-Aware Support Companion

Boundary Objects

Boundary Object	Description
ChatInterface	Text-based interface used by the User to send messages to and receive responses from the EmotionAwareSupportCompanion. The ChatInterface displays conversation history and supports controls such as "Send", "End Session", and quick-access buttons.

HomeScreen	Initial screen presented when the User opens the application. The HomeScreen provides access to Start Conversation, Past Sessions, Privacy Settings, Consent Information, and Feedback options.
ConsentScreen	Screen used for displaying the consent and data-use policy to the User. The ConsentScreen is shown when the User first starts a conversation or explicitly requests to view consent information. It collects the User's agreement.
PrivacySettingsPage	Screen used by the User to review and update PrivacySettings. The PrivacySettingsPage presents options such as "store conversation history," "anonymise logs," and "retention period," and provides controls to save the chosen options.
SessionHistoryView	Screen used for viewing previous ConversationSessions. The SessionHistoryView presents a list of sessions with dates and short summaries, and allows the User to select a particular session to inspect.
FeedbackFormUI	Interface used for collecting feedback from the User at the end of a session or on demand. The FeedbackFormUI displays Likert-scale questions and an optional text field, and forwards the submitted feedback to the appropriate control object.

Table 12 Showing Table – Boundary objects for the Emotion-Aware Support Companion

Control Objects (Table 5-4 Style)

Control Object	Description
InitiateConversationControl	Coordinates UC01 – Initiate Conversation. This control object manages the creation of a new ConversationSession when the User selects *Start Conversation* on the HomeScreen. It triggers the display of the ConsentScreen, processes the User's consent decision via the relevant entity objects, and transitions the system to the ChatInterface once consent has been granted.
SendMessageControl	Coordinates UC02 – Send Message for Analysis. This control object manages the submission of a User Message from the ChatInterface, associates the message with the active ConversationSession, and forwards the message text to the emotion analysis component for processing, while ensuring that logging complies with the User's PrivacySettings.
GenerateResponseControl	Coordinates UC03 – Generate Supportive Response. This control object receives an EmotionRecord resulting from message analysis and selects an appropriate response strategy based on emotion, recent context, and safety rules. It constructs the system's supportive Message, ensures compliance with ethical guardrails, and forwards the response to the ChatInterface for display and storage.
CrisisSupportControl	Coordinates UC04 – Provide Crisis Support Suggestion. This control object evaluates EmotionRecord and message content

	for high-risk indicators. When a potential crisis is detected, it raises a CrisisFlag, selects a crisis-oriented support message, and presents appropriate guidance or external support resources to the User without providing clinical diagnoses or harmful instructions.
ClarificationControl	Coordinates UC05 – Request Clarification. This control object is invoked when emotion analysis produces low confidence or conflicting results. It generates a neutral clarification prompt, presents it to the User via the ChatInterface, and manages the resubmission of any follow-up message for renewed emotion analysis.
ConsentInformationControl	Coordinates UC06 – View Consent Information. This control object retrieves the current consent and data-use policy text and displays it on the ConsentScreen. It records that the information was viewed and manages any updates to the ConsentRecord associated with the UserProfile.
PrivacyPreferencesControl	Coordinates UC07 – Manage Privacy Preferences. This control object retrieves existing PrivacySettings, presents configurable options on the PrivacySettingsPage, validates user-submitted changes, and stores updated settings securely. It ensures that subsequent session handling and data retention behaviour conform to the revised preferences.

EndConversationControl	Coordinates UC08 – End Conversation Session. This control object finalises an active ConversationSession when the User selects *End Session*. It applies the User's privacy and retention rules, marks the session status as closed, prevents further message submission, and returns the User to the HomeScreen.
SessionHistoryControl	Coordinates UC09 – View Conversation History. This control object retrieves stored ConversationSession records permitted by the User's PrivacySettings, generates session summaries, and presents selectable histories on the SessionHistoryView without modifying original records.
FeedbackControl	Coordinates UC10 – Provide Session Feedback. This control object manages the presentation of the FeedbackFormUI, validates submitted ratings and comments, creates a FeedbackEntry, and stores it in an anonymised or pseudonymised manner for later evaluation of system usability and perceived empathy.

Table 13 Showing Control Objects

UML Class / Object Diagram

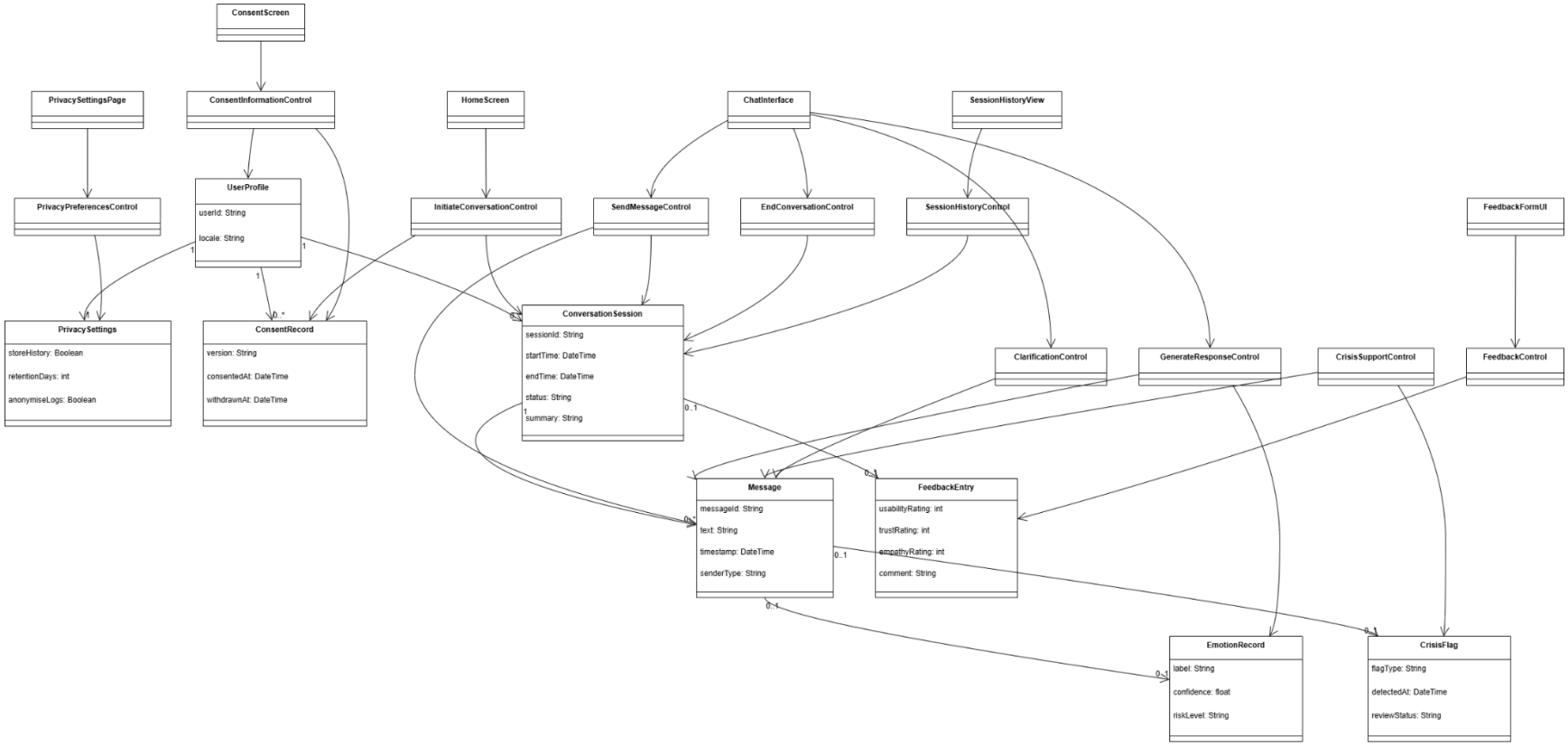

Figure 1 Showing UML Class / Object Diagram

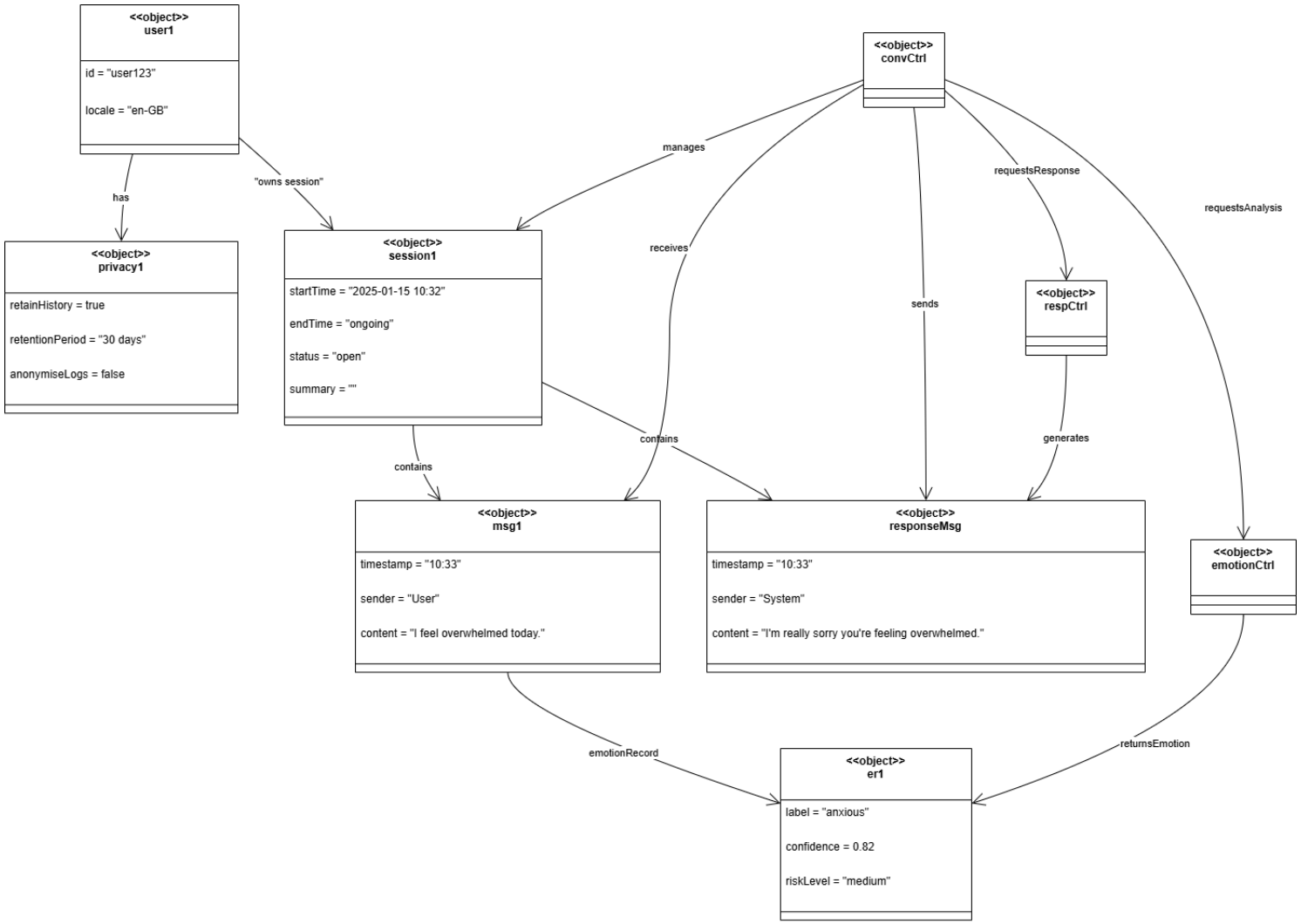

Figure 2 Showing UML Diagram

Objects Grouped into Subsystems

Subsystem	Boundary Objects (A2)	Control Objects (A2)	Entity Objects (A2)
User Interface	HomeScreen, ChatInterface, ConsentScreen, PrivacySettingsPage, SessionHistoryView, FeedbackFormUI		UIState
Conversation Subsystem		ConversationController, SessionHistoryControl	ConversationSession, Message
Emotion & Safety Subsystem		EmotionAnalysisController, CrisisManagerControl, ResponseGeneratorControl	EmotionRecord, CrisisFlag
Consent & Privacy Subsystem		ConsentManagerControl, PrivacySettingsControl	ConsentRecord, PrivacySettings, UserProfile
Data Layer			FeedbackEntry
Audit & Logging			AuditLogEntry

Table 14: Showing Subsystem Descriptions

Subsystem Descriptions

User Interface Subsystem

The User Interface subsystem provides the primary point of interaction between the user and the Emotion-Aware Support Companion. It is responsible for presenting system functionality through screens and controls such as the HomeScreen, ChatInterface, ConsentScreen, PrivacySettingsPage, SessionHistoryView, and FeedbackFormUI. This subsystem collects user input, displays system responses, and routes user actions to the appropriate control subsystems. In addition, the subsystem maintains a UIState entity, which represents transient user interface state such as the currently active screen, selected session, and pending user actions. In accordance with good architectural practice, the User Interface subsystem contains no business logic related to emotion analysis, privacy enforcement, or data persistence, thereby maintaining low coupling with internal system functionality (Alkhomsan, Baslyman, & Alshayeb, 2025).

Conversation Subsystem

The Conversation Subsystem manages the lifecycle and flow of interactions between the user and the Emotion-Aware Support Companion. It is responsible for creating, maintaining, and closing ConversationSession entities, as well as coordinating the exchange of Message entities during active sessions. Control objects such as the ConversationController and SessionHistoryControl ensure that messages are properly associated with sessions and that session history can be retrieved when permitted. This subsystem focuses exclusively on conversational control and coordination, delegating emotion inference, privacy enforcement, and long-term data retention decisions to other subsystems (Fogli & Piccinno, 2005).

Emotion & Safety Subsystem

The Emotion & Safety Subsystem is responsible for analysing user-generated text to infer emotional states and detect potential safety or crisis indicators. Using control objects such as the EmotionAnalysisController, ResponseGeneratorControl, and CrisisManagerControl, this subsystem generates EmotionRecord entities and flags high-risk content through CrisisFlag entities when necessary (Negoitescu, 2025). The subsystem supports empathetic response generation and safety escalation while remaining isolated from consent management and data retention responsibilities. This separation ensures that emotion analysis and safety logic can evolve independently of privacy and conversation management concerns.

Consent & Privacy Subsystem

The Consent & Privacy Subsystem manages user consent decisions and privacy preferences across the system. It records and maintains ConsentRecord and PrivacySettings entities associated with a UserProfile, and acts as the primary entry point for privacy-related user actions such as consent withdrawal or preference updates. Control objects within this subsystem coordinate privacy-related workflows and invoke the Privacy Policy Engine when enforcement actions are required. By centralising consent management within a dedicated subsystem, the architecture ensures consistent handling of privacy rules and clear separation from conversational and analytical logic (Salutari, 2024).

Privacy Policy Engine

The Privacy Policy Engine is a specialised subsystem responsible for enforcing privacy policies through centralised policy evaluation and repository control. It evaluates consent status and retention rules derived from PrivacySettings and ConsentRecord entities and determines the appropriate enforcement actions for affected data artifacts. When triggered, the engine initiates deletion or irreversible anonymisation of ConversationSession, Message, EmotionRecord, and

FeedbackEntry entities (Salutari, 2024). Centralising privacy enforcement within this subsystem minimises coupling, prevents inconsistent privacy handling, and ensures that privacy obligations are applied uniformly across the system.

Data Layer

The Data Layer provides persistent storage services for system data, including FeedbackEntry entities and persisted instances of ConversationSession, Message, and EmotionRecord. It functions as a repository-style subsystem that stores and retrieves data but does not make policy or business decisions (Chinnaraju, 2025). All deletion and anonymisation actions are performed only when explicitly instructed by authorised subsystems such as the Privacy Policy Engine, ensuring that data persistence concerns are clearly separated from system logic and privacy enforcement responsibilities.

Audit & Logging Subsystem

The Audit & Logging subsystem records privacy-related and system-level actions in a privacy-preserving manner. It manages AuditLogEntry entities that capture events such as consent withdrawal, data deletion, and anonymisation actions without storing sensitive conversational or emotional content (Chinnaraju, 2025). This subsystem supports accountability and traceability while complying with privacy requirements, ensuring that system actions can be reviewed or verified without reintroducing personal or sensitive data into the system.

Subsystem Identification (Heuristics Reference)

Subsystems were identified using the heuristics described on page 253 of the textbook. Objects were grouped by functional relatedness, with conversation lifecycle objects kept together, analytical objects isolated for emotion and safety processing, and consent-related objects grouped to centralise privacy management (Bruegge & Dutoit, 2010). Shared persistent data was assigned to a repository-style Data Layer to minimise cross-subsystem dependencies,

while privacy enforcement responsibilities were isolated into a dedicated Privacy Policy Engine to prevent conversational logic from directly controlling retention or deletion behaviour. This decomposition minimises coupling, clarifies subsystem responsibilities, and supports the application of architectural tactics such as centralised privacy enforcement.

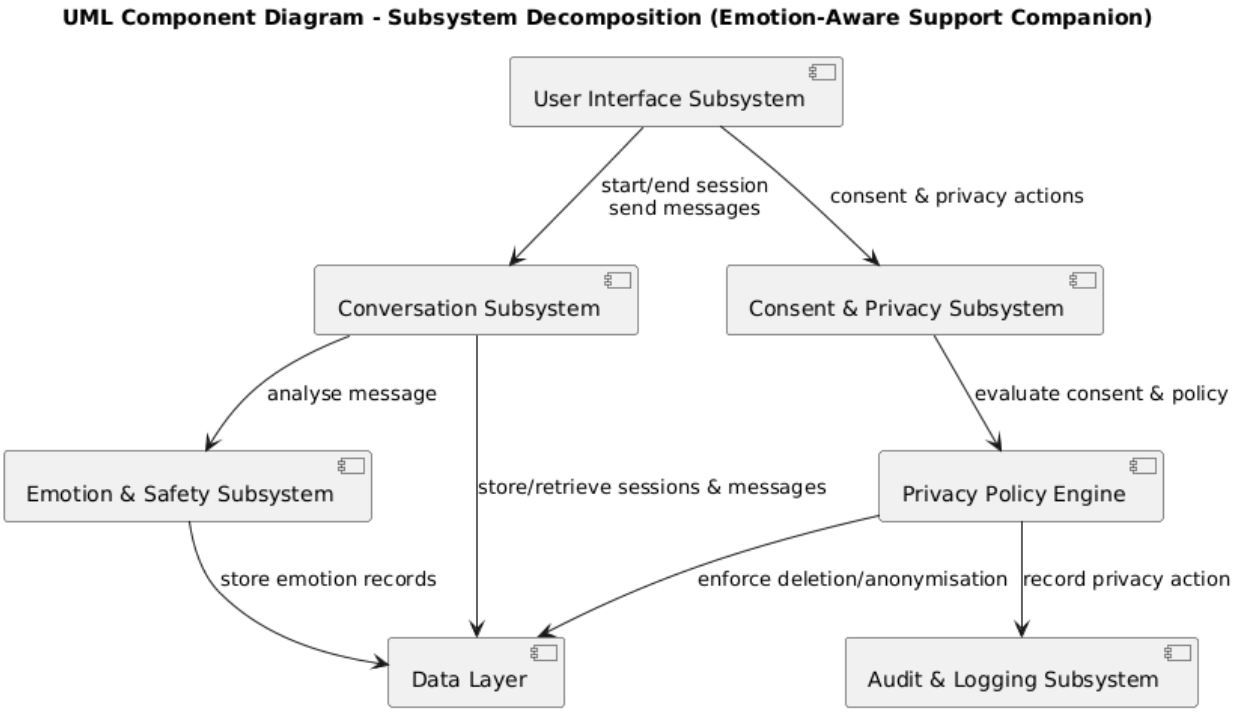

Figure 3 Showing UML Component Diagram – Subsystem Decomposition (Emotion-Aware Support Companion).

Privacy is a critical quality attribute for the Emotion-Aware Support Companion because the system processes sensitive emotional text, inferred emotional states, and user feedback. The architecture must ensure that user data is collected, stored, and deleted strictly according to explicit consent and retention policies, while preventing unauthorised access or unintended data persistence.

System Quality Attribute: Privacy

Scenario Element	Description
Source	User
Stimulus	The User withdraws consent for data retention or requests deletion of previously stored conversation data.
Artifact	Consent & Privacy Subsystem, including the Conversation Storage component and associated data stores containing ConversationSession, Message, EmotionRecord, and FeedbackEntry objects.
Environment	Normal system operation during an active or previously completed conversation session.
Response	The system validates the User's request, identifies all data associated with the UserProfile that is subject to the withdrawn consent, and immediately deletes or anonymises all affected conversation sessions, messages, inferred emotion records, and feedback entries according to the configured privacy policy. The system records the action in a privacy-preserving audit log and informs the User that the request has been completed.
Response Measure	All affected data are deleted or irreversibly anonymised within **2 seconds** of the consent withdrawal request, with **100% coverage** of stored records, and the User receives confirmation of completion without exposing any residual personal or emotional data.

Table 15 Showing System Quality Attribute: Privacy

Privacy Tactic: Policy-Driven Data Retention and Deletion

To support the privacy scenario in which a user withdraws consent for data retention, the Emotion-Aware Support Companion applies a policy-driven data retention and deletion tactic. This tactic centralises privacy enforcement within a dedicated component responsible for interpreting consent and privacy policies and applying them consistently across all stored user data. When a consent withdrawal or retention-expiration event occurs, the privacy enforcement component evaluates the applicable PrivacySettings and ConsentRecord associated with the UserProfile. Based on these policies, the component identifies all related data artifacts, including ConversationSession, Message, EmotionRecord, and FeedbackEntry objects. The tactic ensures that these artifacts are either permanently deleted or irreversibly anonymised according to the defined policy, without requiring direct intervention from other subsystems (Bruegge & Dutoit, 2010).

By separating privacy enforcement from conversational logic and emotion analysis, this tactic reduces coupling between subsystems and ensures that privacy requirements are applied uniformly and reliably. Additionally, the tactic supports traceability by recording the completion of privacy actions in a privacy-preserving audit log, enabling accountability without retaining sensitive user data (Bruegge & Dutoit, 2010).

This architectural approach ensures that privacy obligations are met promptly, consistently, and measurably, even as the system evolves or additional subsystems are introduced.

UML Component Diagram — Privacy Tactic

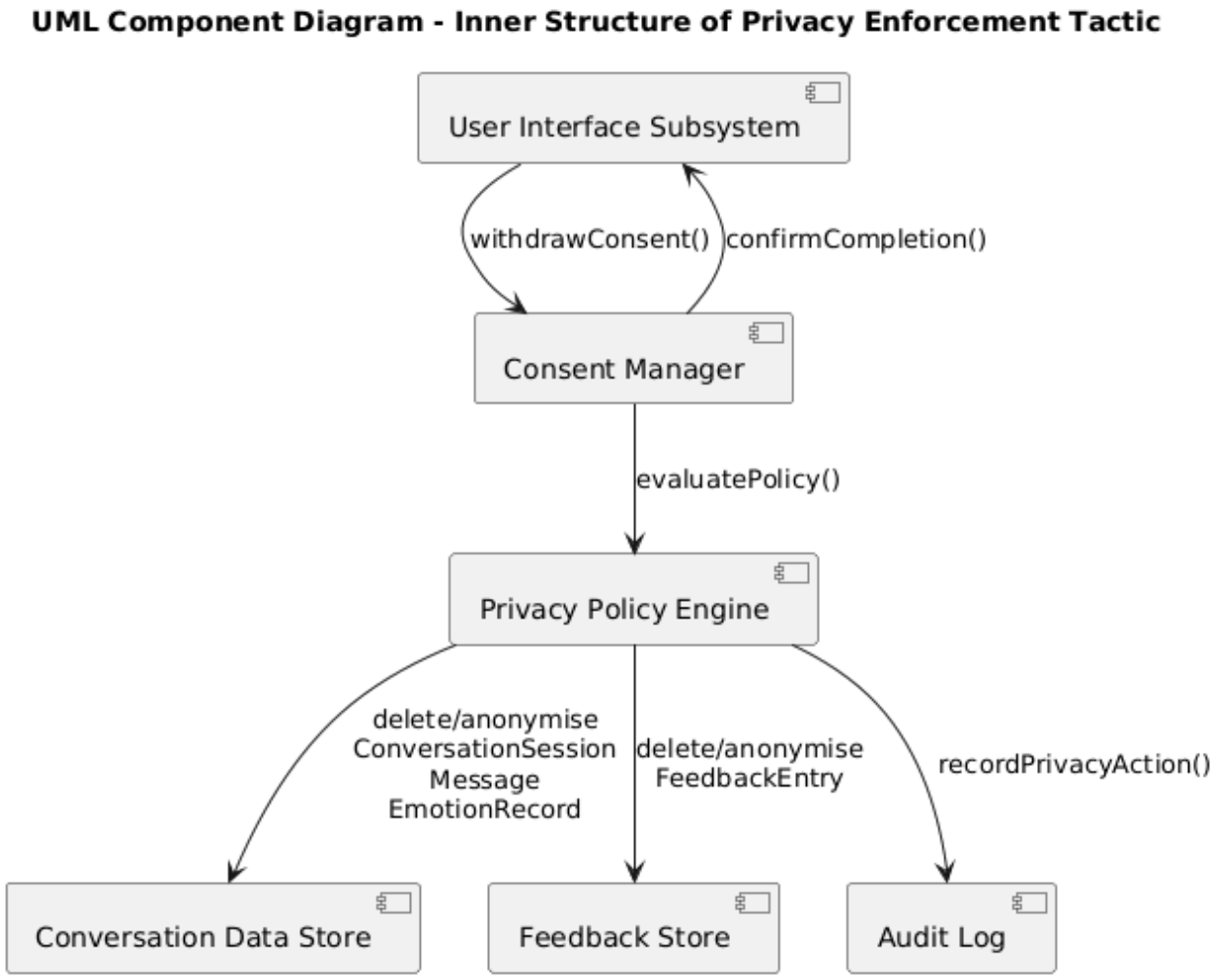

Figure 4 Showing UML component diagram illustrating the policy-driven data retention and deletion tactic. The diagram shows how consent withdrawal triggers privacy policy evaluation and enforcement across subsystems, ensuring timely deletion or anonymisation of sensitive

Subsystem Responsibility Table

Subsystem / Component	Description
User Interface Subsystem	Provides the primary interaction point between the user and the Emotion-Aware Support Companion. It presents system functionality through

	HomeScreen, ChatInterface, ConsentScreen, PrivacySettingsPage, SessionHistoryView, and FeedbackFormUI. The subsystem manages UIState entities representing view state, form input state, and navigation context. It collects user input, displays system responses, and routes user actions to appropriate control subsystems. It contains no business logic related to emotion analysis, privacy enforcement, or data persistence.
Conversation Subsystem	Manages the lifecycle and flow of user interactions. It creates, maintains, and closes ConversationSession entities and coordinates the exchange of Message entities during active sessions. Control objects ensure correct session association and history retrieval. This subsystem focuses solely on conversational coordination and delegates emotion inference, privacy enforcement, and persistence decisions to other subsystems.
Emotion & Safety Subsystem	Analyses user-generated text to infer emotional states and detect safety or crisis indicators. It produces EmotionRecord entities and raises CrisisFlag entities when high-risk patterns are detected. The subsystem supports empathetic response generation and safety escalation while remaining isolated from consent, retention, and storage responsibilities.
Consent & Privacy Subsystem	Manages user consent and privacy preferences. It records ConsentRecord and PrivacySettings entities associated with a UserProfile. This subsystem acts as the entry point for privacy-related user actions such as consent withdrawal and preference updates, and coordinates enforcement workflows by invoking the Privacy Policy Engine.

Privacy Policy Engine	Enforces privacy policies through policy-driven data retention and deletion mechanisms. It evaluates RetentionPolicy entities derived from PrivacySettings and ConsentRecord data and determines required enforcement actions. The engine identifies affected ConversationSession, Message, EmotionRecord, and FeedbackEntry entities and triggers deletion or irreversible anonymisation. Centralising enforcement here ensures consistent and auditable privacy handling.
Data Layer	Provides persistent storage for system entities including ConversationSession, Message, EmotionRecord, FeedbackEntry, and AuditLogEntry. It acts as a repository-style subsystem and performs storage, retrieval, deletion, and anonymisation only when instructed by authorised subsystems. It does not make business or policy decisions.
Audit & Logging Subsystem	Records privacy-related and system-level actions using AuditLogEntry entities. It captures events such as consent withdrawal, data deletion, and anonymisation without storing sensitive conversational or emotional content. This subsystem supports accountability and traceability while preserving privacy guarantees.

Table 16 Showing Subsystem Responsibility Table

Expanded Class/Object Model

Modelling Conventions Used

Item	**Convention**
Access Modifiers	private (−), public (+), protected (#)

Data Types	int, double, String, boolean, DateTime, List<T>
Method Format	+ methodName(parameter: Type): ReturnType

1. UserProfile

UserProfile
-long userId -String username -String email -DateTime createdAt
+void updateEmail(String newEmail) +void deactivate()

2. ConversationSession

ConversationSession
-long sessionId -long userId -DateTime startTime -DateTime endTime -String status
+void start() +void close() +void addMessage(Message m) +List getMessages()

3. Message

Message
-long messageId -long sessionId -String sender -String content -DateTime timestamp
+boolean validate() +void anonymise()

4. EmotionRecord

EmotionRecord
-long emotionId -long messageId -String emotionType -double confidence -DateTime createdAt
+boolean isHighRisk()

5. CrisisFlag

CrisisFlag
-long flagId -long emotionId -String riskLevel -DateTime createdAt
+void escalate()

6. ConsentRecord

ConsentRecord
-long consentId -long userId -boolean granted -DateTime grantedAt
+void withdraw()

7. PrivacySettings

PrivacySettings
-long settingsId -long userId -int retentionDays -boolean allowAnalytics
+void updateRetention(int days)

8. FeedbackEntry

FeedbackEntry
-long feedbackId -long sessionId -int rating -String comments
+boolean validate()

9. AuditLogEntry

AuditLogEntry
-long logId -String action -String actor -DateTime timestamp
+void record()

10. RetentionPolicy

RetentionPolicy
-long policyId -int retentionDays -boolean anonymiseAfter
+boolean evaluate()

Identification of Persistent Entity Objects

This section identifies which entity objects from the expanded class model must be stored in a relational database to support system functionality, auditing, and privacy compliance.

Following the textbook guidance, an object is persistent if it:

i. Must survive system shutdown

ii. Is shared across multiple sessions

iii. Is required for auditing, reporting, or legal compliance

iv. Represents a business or domain concept rather than a temporary runtime state

Persistent vs Non-Persistent Entities

Entity Object	Persistent?	Justification
UserProfile	Yes	Required to associate sessions, consent, and privacy settings with a user over time.
ConversationSession	Yes	Needed to retrieve conversation history, apply retention policies, and support audits.
Message	Yes	Core conversational data required for context, analysis, and possible anonymisation or deletion.
EmotionRecord	Yes	Used for evaluation, safety review, and to support analytics and system improvement.
CrisisFlag	Yes	Required for safety audits, risk tracking, and escalation verification.
ConsentRecord	Yes	Legal requirement to prove when and how consent was granted or withdrawn.
PrivacySettings	Yes	Determines how long data is retained and whether it is anonymised.

FeedbackEntry	Yes	Required for usability evaluation and system improvement metrics.
AuditLogEntry	Yes	Required for accountability, traceability, and compliance verification.
RetentionPolicy	Yes	Represents configurable system rules applied across all users.
UIState	No	Runtime-only object representing transient screen state; not part of business data.

Table 17 Showing Persistent vs Non-Persistent Entities

Relational Database Mapping (Object → Table)

UserProfile → USER_PROFILE

Column	**Data Type**	**Key**	**Description**
user_id	BIGINT	PK	Unique user identifier
username	VARCHAR(100)		Display name
email	VARCHAR(150)		User email
created_at	DATETIME		Account creation timestamp

ConversationSession → CONVERSATION_SESSION

Column	Data Type	Key	Description
session_id	BIGINT	PK	Unique session identifier
user_id	BIGINT	FK → USER_PROFILE.user_id	Owner
start_time	DATETIME		Session start
end_time	DATETIME		Session end
status	VARCHAR(20)		OPEN / CLOSED

Message → MESSAGE

Column	Data Type	Key	Description
message_id	BIGINT	PK	Unique message
session_id	BIGINT	FK → CONVERSATION_SESSION.session_id	Parent session
sender	VARCHAR(20)		USER / SYSTEM
content	TEXT		Message body
timestamp	DATETIME		Time sent

EmotionRecord → EMOTION_RECORD

Column	Data Type	Key	Description
emotion_id	BIGINT	PK	Unique emotion record
message_id	BIGINT	FK → MESSAGE.message_id	Source message
emotion_type	VARCHAR(50)		sad, anxious, etc.
confidence	DOUBLE		Confidence score
created_at	DATETIME		Analysis time

CrisisFlag → CRISIS_FLAG

Column	Data Type	Key	Description
flag_id	BIGINT	PK	Unique flag
emotion_id	BIGINT	FK → EMOTION_RECORD.emotion_id	Source
risk_level	VARCHAR(20)		LOW / HIGH
created_at	DATETIME		Flag time

ConsentRecord → CONSENT_RECORD

Column	Data Type	Key	Description
consent_id	BIGINT	PK	Unique record

user_id	BIGINT	FK → USER_PROFILE.user_id	Owner
granted	BOOLEAN		TRUE/FALSE
granted_at	DATETIME		Timestamp

PrivacySettings → PRIVACY_SETTINGS

Column	**Data Type**	**Key**	**Description**
settings_id	BIGINT	PK	Unique record
user_id	BIGINT	FK → USER_PROFILE.user_id	Owner
retention_days	INT		Days
allow_analytics	BOOLEAN		TRUE/FALSE

FeedbackEntry → FEEDBACK_ENTRY

Column	**Data Type**	**Key**	**Description**
feedback_id	BIGINT	PK	Unique entry
session_id	BIGINT	FK → CONVERSATION_SESSION.session_id	Source
rating	INT		1–5
comments	TEXT		Optional

AuditLogEntry → AUDIT_LOG

Column	Data Type	Key	Description
log_id	BIGINT	PK	Unique log
action	VARCHAR(100)		Event
actor	VARCHAR(100)		System/User
timestamp	DATETIME		Time

RetentionPolicy → RETENTION_POLICY

Column	Data Type	Key	Description
policy_id	BIGINT	PK	Unique policy
retention_days	INT		Days
anonymise_after	BOOLEAN		TRUE/FALSE

Mapping Associations

5.2.1 One-to-Many (1..*) Mappings

UserProfile (1) → ConversationSession (many)

- Implemented by: CONVERSATION_SESSION.user_id (FK)

ConversationSession (1) → Message (many)

- Implemented by: MESSAGE.session_id (FK)

Message (1) → EmotionRecord (many/usually one)

- Implemented by: EMOTION_RECORD.message_id (FK)

EmotionRecord (1) → CrisisFlag (0..1 or many depending on design)

- Implemented by: CRISIS_FLAG.emotion_id (FK)

UserProfile (1) → ConsentRecord (many)

- Implemented by: CONSENT_RECORD.user_id (FK)

5.2.2 One-to-One (1..1) Mapping

UserProfile (1) → PrivacySettings (1)

- Implemented by: PRIVACY_SETTINGS.user_id as a foreign key with UNIQUE constraint
- This ensures each user can have only one settings record.

Many-to-Many Mapping

No many-to-many relationship is required in the current system design. All relationships are adequately represented using one-to-many and one-to-one mappings. Therefore, no association (junction) table is required.

Mapping Summary Table (Class Attribute → Table Column)

This table provides a consolidated mapping view of object attributes into relational schema columns, including key constraints.

Class.Attribute	**Table.Column**	**SQL Type**	**Key/Constraint**
UserProfile.userId	USER_PROFILE.user_id	BIGINT	PK, NOT NULL
UserProfile.username	USER_PROFILE.username	VARCHAR(100)	NOT NULL
UserProfile.email	USER_PROFILE.email	VARCHAR(150)	UNIQUE, NOT NULL
UserProfile.createdAt	USER_PROFILE.created_at	DATETIME	NOT NULL
PrivacySettings.settingsId	PRIVACY_SETTINGS.settings_id	BIGINT	PK, NOT NULL
PrivacySettings.userId	PRIVACY_SETTINGS.user_id	BIGINT	FK, UNIQUE, NOT NULL
PrivacySettings.retentionDays	PRIVACY_SETTINGS.retention_days	INT	NOT NULL
PrivacySettings.allowAnalytics	PRIVACY_SETTINGS.allow_analytics	BOOLEAN	NOT NULL
ConsentRecord.consentId	CONSENT_RECORD.consent_id	BIGINT	PK, NOT NULL

ConsentRecord.userId	CONSENT_RECORD.user_id	BIGINT	FK, NOT NULL
ConsentRecord.granted	CONSENT_RECORD.granted	BOOLEAN	NOT NULL
ConsentRecord.grantedAt	CONSENT_RECORD.granted_at	DATETIME	NOT NULL
ConversationSession.sessionId	CONVERSATION_SESSION.session_id	BIGINT	PK, NOT NULL
ConversationSession.userId	CONVERSATION_SESSION.user_id	BIGINT	FK, NOT NULL
ConversationSession.startTime	CONVERSATION_SESSION.start_time	DATETIME	NOT NULL
ConversationSession.endTime	CONVERSATION_SESSION.end_time	DATETIME	NULL
ConversationSession.status	CONVERSATION_SESSION.status	VARCHAR(20)	NOT NULL
Message.messageId	MESSAGE.message_id	BIGINT	PK, NOT NULL
Message.sessionId	MESSAGE.session_id	BIGINT	FK, NOT NULL

Message.sender	MESSAGE.sender	VARCHAR(20)	NOT NULL
Message.content	MESSAGE.content	TEXT	NOT NULL
Message.timestamp	MESSAGE.timestamp	DATETIME	NOT NULL
EmotionRecord.emotionId	EMOTION_RECORD.emotion_id	BIGINT	PK, NOT NULL
EmotionRecord.messageId	EMOTION_RECORD.message_id	BIGINT	FK, NOT NULL
EmotionRecord.emotionType	EMOTION_RECORD.emotion_type	VARCHAR(50)	NOT NULL
EmotionRecord.confidence	EMOTION_RECORD.confidence	DOUBLE	NOT NULL
EmotionRecord.createdAt	EMOTION_RECORD.created_at	DATETIME	NOT NULL
CrisisFlag.flagId	CRISIS_FLAG.flag_id	BIGINT	PK, NOT NULL
CrisisFlag.emotionId	CRISIS_FLAG.emotion_id	BIGINT	FK, NOT NULL
CrisisFlag.riskLevel	CRISIS_FLAG.risk_level	VARCHAR(20)	NOT NULL

CrisisFlag.createdAt	CRISIS_FLAG.created_at	DATETIME	NOT NULL
FeedbackEntry.feedbackId	FEEDBACK_ENTRY.feedback_id	BIGINT	PK, NOT NULL
FeedbackEntry.sessionId	FEEDBACK_ENTRY.session_id	BIGINT	FK, NOT NULL
FeedbackEntry.rating	FEEDBACK_ENTRY.rating	INT	NOT NULL
FeedbackEntry.comments	FEEDBACK_ENTRY.comments	TEXT	NULL
AuditLogEntry.logId	AUDIT_LOG.log_id	BIGINT	PK, NOT NULL
AuditLogEntry.action	AUDIT_LOG.action	VARCHAR(100)	NOT NULL
AuditLogEntry.actor	AUDIT_LOG.actor	VARCHAR(100)	NOT NULL
AuditLogEntry.timestamp	AUDIT_LOG.timestamp	DATETIME	NOT NULL
RetentionPolicy.policyId	RETENTION_POLICY.policy_id	BIGINT	PK, NOT NULL
RetentionPolicy.retentionDays	RETENTION_POLICY.retention_days	INT	NOT NULL

RetentionPolicy.anonymiseAfter	RETENTION_POLICY.anonymise_after	BOOLEAN	NOT NUL

Table 18 Showing Mapping Summary Table

API Design

Entity Summary

ConversationSession represents a single interaction session between a user and the Emotion-Aware Support Companion.

ConversationSession (Expanded Object)

Class: ConversationSession

Attribute	Type	Access	Description
sessionId	BIGINT	− private	Unique session identifier
userId	BIGINT	− private	Owner (FK → UserProfile)
startTime	DateTime	− private	Session start time
endTime	DateTime	− private	Session end time
status	String	− private	OPEN / CLOSED
messages	List	# protected	Messages in this session

Table 19 Showing

Methods

Method Signature	Return Type	Description
+ startSession(userId: BIGINT): ConversationSession	ConversationSession	Creates and opens a new session
+ closeSession(): void	void	Marks session as CLOSED and sets endTime
+ addMessage(msg: Message): void	void	Adds a message to the session
+ getMessages(): List	List	Retrieves all messages in the session
+ isActive(): boolean	boolean	Returns true if status is OPEN

Table 20 Showing Methods

REST API Design (CRUD)

/api/v1/sessions

1.Create Session

POST /api/v1/sessions

Content-Type: application/json

```
{
  "userId": 102,
  "status": "OPEN"
}
```

2.0Response

```
{
  "sessionId": 501,
  "userId": 102,
  "startTime": "2026-02-04T10:32:11Z",
  "status": "OPEN"
}
```

2.2 Get Session by ID

GET /api/v1/sessions/501

2.3 List User Sessions

GET /api/v1/users/102/sessions

2.4 Close Session

PUT /api/v1/sessions/501/close

2.5 Delete Session (Consent Withdrawal)

DELETE /api/v1/sessions/501

3. UserProfile

3.1 Create User

POST /api/v1/users

3.2 Get User by ID

GET /api/v1/users/{userId}

3.3 List Users

GET /api/v1/users

3.4 Update User

PUT /api/v1/users/{userId}

3.5 Delete User

DELETE /api/v1/users/{userId}

4. Message

4.1 Create Message

POST /api/v1/sessions/{sessionId}/messages

4.2 Get Message by ID

GET /api/v1/messages/{messageId}

4.3 List Messages in Session

GET /api/v1/sessions/{sessionId}/messages

4.4 Update Message

PUT /api/v1/messages/{messageId}

4.5 Delete Message

DELETE /api/v1/messages/{messageId}

5. EmotionRecord

5.1 Create Emotion Record

POST /api/v1/messages/{messageId}/emotions

5.2 Get Emotion Record by ID

GET /api/v1/emotions/{emotionId}

5.3 List Emotion Records for Message

GET /api/v1/messages/{messageId}/emotions

5.4 Update Emotion Record

PUT /api/v1/emotions/{emotionId}

5.5 Delete Emotion Record

DELETE /api/v1/emotions/{emotionId}

6. CrisisFlag

6.1 Create Crisis Flag

POST /api/v1/emotions/{emotionId}/crisis-flags

6.2 Get Crisis Flag by ID

GET /api/v1/crisis-flags/{flagId}

6.3 List Crisis Flags for Emotion

GET /api/v1/emotions/{emotionId}/crisis-flags

6.4 Update Crisis Flag

PUT /api/v1/crisis-flags/{flagId}

6.5 Delete Crisis Flag

DELETE /api/v1/crisis-flags/{flagId}

7. ConsentRecord

7.1 Create Consent Record

POST /api/v1/users/{userId}/consents

7.2 Get Consent Record by ID

GET /api/v1/consents/{consentId}

7.3 List User Consent Records

GET /api/v1/users/{userId}/consents

7.4 Update Consent Record

PUT /api/v1/consents/{consentId}

7.5 Delete Consent Record

DELETE /api/v1/consents/{consentId}

8. PrivacySettings

8.1 Create Privacy Settings

POST /api/v1/users/{userId}/privacy-settings

8.2 Get Privacy Settings

GET /api/v1/users/{userId}/privacy-settings

8.3 Update Privacy Settings

PUT /api/v1/users/{userId}/privacy-settings

8.4 Delete Privacy Settings

DELETE /api/v1/users/{userId}/privacy-settings

9. FeedbackEntry

9.1 Create Feedback Entry

POST /api/v1/sessions/{sessionId}/feedback

9.2 Get Feedback by ID

GET /api/v1/feedback/{feedbackId}

9.3 List Feedback for Session

GET /api/v1/sessions/{sessionId}/feedback

9.4 Update Feedback

PUT /api/v1/feedback/{feedbackId}

9.5 Delete Feedback

DELETE /api/v1/feedback/{feedbackId}

10. AuditLogEntry

10.1 Create Audit Log

POST /api/v1/audit-logs

10.2 Get Audit Log by ID

GET /api/v1/audit-logs/{logId}

10.3 List Audit Logs

GET /api/v1/audit-logs

10.4 Delete Audit Log

DELETE /api/v1/audit-logs/{logId}

11. RetentionPolicy

11.1 Create Retention Policy

POST /api/v1/retention-policies

11.2 Get Retention Policy by ID

GET /api/v1/retention-policies/{policyId}

11.3 List Retention Policies

GET /api/v1/retention-policies

11.4 Update Retention Policy

PUT /api/v1/retention-policies/{policyId}

11.5 Delete Retention Policy

DELETE /api/v1/retention-policies/{policyId}

GraphQL API Design

Type Definition

```
type ConversationSession {
  sessionId: ID!
  userId: ID!
  startTime: String!
  endTime: String
  status: String!
}
```

Queries

```
query GetSession($id: ID!) {
  session(id: $id) {
    sessionId
    status
    startTime
  }
}
query GetUserSessions($userId: ID!) {
  sessionsByUser(userId: $userId) {
    sessionId
    status
  }
}
```

Mutations

```
mutation CreateSession($userId: ID!) {
  createSession(userId: $userId) {
    sessionId
    status
  }
}
mutation CloseSession($id: ID!) {
  closeSession(sessionId: $id) {
    status
    endTime
  }
}
mutation DeleteSession($id: ID!) {
```

```
  deleteSession(sessionId: $id)
}
```

Conclusion

The Emotion-Aware Support Companion has been modelled using structured use case analysis and object-oriented design heuristics. The ten use cases define the system's behavioural requirements, while the identified entity, boundary, and control objects translate these behaviours into a coherent object-oriented architecture. The UML class diagram illustrates the overall structure, and the object diagram provides a concrete runtime example of interactions during a conversation. Together, these artefacts offer a complete and traceable design foundation for further development of the system**.** This report expanded the Emotion-Aware Support

Companion's object model by specifying full attributes and method signatures for each object, including explicit data types and access levels. It then identified which entity objects must be persisted to support core system functions such as conversation history, consent tracking, privacy enforcement, safety auditing, and user feedback analysis. Using a forward-engineering approach consistent with the textbook, each persistent class was mapped to a relational table with appropriate primary keys, foreign keys, and constraints to preserve object relationships. The resulting schema ensures traceable, policy-compliant storage while maintaining clear separation between transient runtime state and durable system records. Overall, the expanded class model and relational mappings provide a complete, implementation-ready foundation for building and evaluating the system in a robust and privacy-aware manner.

Perspective Thoughts and Reflections for Future Directions

i. Incorporating more advanced NLP and multimodal analysis (e.g., tone, behavioral patterns) could improve accuracy in emotion detection and crisis flagging, while reducing false positives.

ii. Expanding API interoperability (REST and GraphQL) and optimizing database performance will support larger user bases, external system integration, and real-time analytics for continuous system improvement.

Reference

Alkhomsan, M. N., Baslyman, M., & Alshayeb, M. (2025). Towards a Better Understanding of User Emotion: An Emotion-Aware Requirements Elicitation Support Tool: EmORE Tool. IEEE Access.

Bruegge, B., & Dutoit, A. H. (2010). Object oriented software engineering: Using UML patterns and Java. Prentice Hall.

Chinnaraju, A. (2025). Emotion-Aware AI for Learning and Organizations: An Affective Computing Framework for Adaptive Human–AI Interaction. INTERNATIONAL JOURNAL OF ENGINEERING DEVELOPMENT AND RESEARCH, 13(4), 441-536.

Fogli, D., & Piccinno, A. (2005). Environments to support context and emotion aware visual interaction. Journal of Visual Languages & Computing, 16(5), 386-405.

Negoitescu, M. C. (2025). Towards Emotionally and Motivationally Aware Intelligent Systems: A Systematic Literature Review (Doctoral dissertation, Delft University of Technology).

Salutari, A. (2024). Harmonizing Users' and System's Requirements in Complex and Resource Intensive Application Domains by a Distributed Hybrid Approach.